CS598 HPC. Homework 4 - Parallel Tridiagonal Solvers

Due: Friday 10/23/26

Objective: Develop distributed-memory tridiagonal matrix solver.

We wish to solve

$$A\underline{u} \;=\; \underline{f}, \tag{1}$$

with $A = \text{tridiag}(c_i, a_i, b_i) \in \mathbb{R}^{n\times n}$, and $n \gg P \gg 1$. We will make a significant simplifying assumption that A is either SPD or diagonally dominant. Under these conditions, *LU* factorization without pivoting will be *stable* and standard LU can be used *in serial.*

Consider

$$\begin{bmatrix} a_1 & b_1 & & & \\ c_2 & a_2 & b_2 & & \\ & c_3 & a_3 & b_3 & \\ & & \ddots & \ddots & \ddots \\ & & & c_n & a_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_n \end{bmatrix}, \tag{2}$$

where

- c_i = subdiagonal ($i = 2, \ldots, n$)
- a_i = diagonal ($i = 1, \ldots, n$)
- b_i = superdiagonal ($i = 1, \ldots, n-1$).

For your P-processor parallel implementation, use the domain-decomposition approach discussed in class, in which we identify $P-1$ separator nodes v_j associated with equation j in the graph equivalent to the tridiagonal matrix A. Enumerating these last leaves P diagonal blocks, A_p, $p = 1, \ldots, P$, each of which is tridiagonal. Upon reordering, the new system reads:

$$\tilde{A} \;:=\; P^T A P \;=\; \left[\begin{array}{cccc|c} A_1 & & & & U_1 \\ & A_2 & & & U_2 \\ & & \ddots & & \vdots \\ & & & A_P & U_P \\ \hline L_1 & L_2 & \cdots & L_P & D \end{array}\right] \tag{3}$$

As discussed in class, for each rank p we have the local data structure,

$$\left[\begin{array}{cccc|cc} a_1 & b_1 & & & u_1 & 0 \\ c_2 & a_2 & \ddots & & u_2 & 0 \\ & \ddots & \ddots & b_{n-1} & \vdots & \vdots \\ & & c_n & a_n & u_n & u_{n2} \\ \hline l_1 & l_2 & \cdots & l_n & S_{11} & S_{12} \\ & & & l_{2n} & S_{21} & S_{22} \end{array}\right]. \tag{4}$$

In addition to vectors $\underline{a}$, $\underline{b}$, and $\underline{c}$, you will need to track $\underline{l}$ and $\underline{u}$, plus the 2×2 Schur complement, which should be initialized to $S_{ij} = a_{n+1}$ for $i = j = 2$, 0 otherwise.

Once these blocks are factorized, you'll need to update the global Schur complement and solve that cooperatively (or redundantly on each rank), so that you can perform the local back substitution, which will be fully parallel as soon as the interface (i.e., separator) values of x_j are known.

As usual, write up a description of your approach, verification, observations, performance results, and analysis. Please include the code in an appendix.